

TG1: Trabalho em Grupo sobre Python

Sistema de Torneio Pokémon

Atenção: para que a avaliação da aprendizagem seja mais eficaz, recomendamos não utilizar inteligência artificial no desenvolvimento deste trabalho. Caso detectemos o uso, o trabalho será zerado.

Objetivo

Durante a disciplina de programação orientada a dados, vocês estão aprendendo a programar em Python, uma linguagem de programação multiparadigma. Neste trabalho, o objetivo é avaliá-lo em relação às partes de Programação Orientada a Objetos (POO), Módulos e Manipulação de Arquivos. O trabalho também será uma forma de exercitar e aplicar os conceitos na prática.

Contexto

Você foi escolhido para integrar um grupo em uma pesquisa universitária, com o intuito de ajudar estudantes a entender os conceitos básicos de Programação Orientada a Objetos por meio de simulações. A ideia principal do trabalho é utilizar um sistema simples, mas familiar para implementar um torneio, para isto os autores escolheram alguns sistemas dos populares jogos da franquia Pokémon, tais como o sistema de batalha, utilização de itens, o sistema de tipos que são utilizados em conjunto ao sistema de golpes e pokémons em geral e o sistema de efeito de status. Devido a alta complexidade que estes sistemas possuem graças a longevidade da franquia, grande parte destes sistemas foram simplificados para fácil implementação das ideias por trás da simulação.

Em resumo, um torneio de Pokémon funciona da seguinte forma. No seu escopo mais micro, o sistema trata de batalhas 1v1 entre dois Pokémon. Estas batalhas são segmentadas em turnos, em que a ordem das ações de cada turno é determinada pelo atributo de velocidade dos Pokémon combatentes. A cada turno, os treinadores devem decidir se usarão um item do próprio inventário ou se comandarão seu Pokémon a atacar. Uma batalha entre dois treinadores só se encerra quando um deles não tiver mais Pokémon não desmaiados.

No escopo macro do sistema, o torneio é composto por treinadores; estes possuem até 6 Pokémon e uma série de itens voltados à cura e à remoção de efeitos. Adentrando um pouco nos sistemas complementares, cada Pokémon possui alguns atributos, Vida, Ataque, Defesa e Velocidade, além de até 4 golpes. Cada golpe tem alguns atributos que são utilizados para determinar se o golpe conectará com o oponente e para o cálculo do dano infligido, além disso, alguns golpes podem também causar efeitos de status ao oponente.

Cada Pokémon e cada Golpe também possuem um Tipo. Este atributo é utilizado no cálculo de dano sofrido. Esta definição é normalmente atribuída via uma tabela de multiplicação entre dois tipos, um pertencente ao Golpe sendo aplicado e o outro ao Pokémon sofrendo o Golpe.

Restrições

- Está permitido o uso de todos os métodos da classe str (string).
- Apenas os módulos **abc**, **sys**, **csv**, **random** e **math** podem ser importados para implementar o sistema.
- Outros métodos, como **len()** que são nativos e não dependem de importação de módulos, estão autorizados ([lista de métodos](#)). **Métodos que não foram apresentados em aula devem ser justificados e explicados por meio de comentários no código.**

Especificações

Você precisa desenvolver um sistema de torneios baseado em batalhas no estilo dos jogos Pokémon que possa ser utilizado como um pacote do Python, contendo dois módulos (Sistema.py e Erros.py) e uma aplicação em Python que utilize este pacote para realizar uma sequência de operações que estão pré-definidas em um arquivo.

Especificações e requisitos do sistema:

- 1) Os atributos da classe devem ser manipulados por meio de setters e getters, e sua acessibilidade deve ser devidamente definida como private, protected ou public.
- 2) Checagens e testes de sanidade devem ser implementados nas classes do sistema para que operações indevidas sejam interceptadas e tratadas na aplicação.
- 3) Use sobrecarga de operadores nas classes quando possível e aplicável.
- 4) Precisa possuir uma classe **Tipo**, para encapsular os seguintes dados:
 - a) Nome do tipo.
 - b) Efetividade (Dict contendo os multiplicadores de dano por tipo de atacante).
- 5) Precisa possuir uma classe abstrata **EfeitoStatus**, que armazena o nome do efeito (máximo de 10 caracteres).

Obs.: Esta classe deve possuir o método abstrato **aplicar()**, que será implementado futuramente pelas classes filhas, as quais definirão os efeitos de status causados por alguns golpes.

- 6) Precisa possuir as subclasses **Queimadura** e **Envenenado** que estendem a classe **EfeitoStatus**, para implementar o envenenamento (dano de 5 pontos) e a queimadura (dano de 3 pontos).
- 7) Precisa possuir a classe **Golpe**, com os seguintes dados:
 - a) Nome do golpe (máximo de 15 caracteres)
 - b) Tipo do golpe
 - c) Poder do golpe (inteiro de valor máximo de 250).
 - d) Acurácia do golpe (float com valor máximo de 1).
 - e) Efeito do golpe.
 - f) Chance de Efeito (float com valor máximo de 1).

Obs.: Para fins de simplificação, utilize a seguinte fórmula para realizar o cálculo de dano infligido pelos golpes durante o combate (para fins de simplificação, assumo que o valor

do Nível será sempre 50):

- **A:** Valor do atributo Ataque do Pokémon Atacante.
- **D:** Valor do atributo Defesa do Pokémon Alvo.
- **Base:** em que A representa o ataque e D, a defesa.

$$\frac{\left[\left(\frac{2 \times \text{Nivel}}{5} \right) \right] \cdot \text{Poder} \cdot \frac{A}{D}}{50} + 2$$

- **Modificador:** Random(0.85, 1) * (TipoAtacante * TipoAlvo)
- **Dano:** Base * Modificador

8) Precisa possuir uma classe abstrata **Item**, que deve respeitar os seguintes requisitos:

- Ter um atributo booleano CuraStatus.
- Possuir os métodos abstratos **pode_usar()** e **usar()**, que serão futuramente implementados pelas classes filhas, sendo estes os itens utilizados durante as batalhas.
- Item** é a classe pai das subclasses **Pocao**, **SuperPocao**, **Antidoto**, **Antiqueimadura** e **CuraTotal**; todas estas estendem a classe **Item** e implementam os seguintes comportamentos:
 - Pocao** e **SuperPocao** curam um Pokémon em 20 e 50 pontos de vida, respectivamente, e só devem ser utilizados se o alvo estiver com menos de 80% e 50% de sua vida máxima;
 - Antidoto**, **Antiqueimadura** e **CuraTotal** curam, respectivamente, os efeitos de status *Envenenado* e *Queimadura*. **CuraTotal**, por sua vez, é genérica e deve remover todos os status.
 - Os itens não devem ser subutilizados, ou seja, deve ser impossível utilizar um item de cura se o Pokémon estiver com a vida cheia, ou um item de remoção de status se o Pokémon não estiver sofrendo o status.

9) Precisa possuir a classe **Acao**, a qual deve possuir um método abstrato **executar()**, que será implementado pelas seguintes subclasses.

- Subclasse **AcaoAtq**, que possui um atributo para armazenar um golpe.
- Subclasse **AcaoItem**, que possui um atributo para armazenar um item.

10) Precisa de uma classe **Pokemon**, para encapsular os seguintes dados:

- Nome do Pokémon (Max. 15 caracteres)
- Tipo do Pokémon.
- Vida máxima (valor inteiro, com limite máximo de 255).
- Vida atual (valor inteiro, com limite máximo de 255).
- Valor do ataque (valor inteiro, com limite máximo de 255).
- Valor da defesa (valor inteiro, com limite máximo de 255).
- Valor de velocidade (valor inteiro, com limite máximo de 255).
- Lista de golpes.
- Lista de estatus atualmente afetando o Pokémon.

Obs.: os itens **C** a **G** devem, dentro do método de inicialização da classe, utilizar as seguintes fórmulas para calcular as estatísticas efetivas (para fins de simplificação, assumo que o valor do nível será sempre 50):

Vida Máxima e Atual: onde **HP** (Health Point) representa o valor base da vida de um Pokémon.

$$\left\lceil \left(\frac{2 \times HP}{100} \right) * Nivel \right\rceil + Nivel + 10$$

Outros: onde **Stat** representa o valor base dos atributos de ataque, defesa ou velocidade.

$$\left\lceil \frac{(2 \cdot Stat) \cdot Nivel}{100} \right\rceil + 5$$

11) Deve conter a classe **Treinador**, que armazena os seguintes dados:

- Nome do treinador (Máximo de 20 caracteres)
- Quantidade de Pokémons (Máximo de 6 Pokémons)
- Lista/dicionário de itens.

Obs.: Esta classe deve possuir um método de escolha aleatória com divisão 70/30, no qual deve escolher uma ação (**Atacar** ou usar **Item**), sendo que cada ação é mutuamente exclusiva. No caso de utilizar um item, a prioridade deve ser dada a itens de cura. Além disso, a classe também pode possuir alguns métodos auxiliares, tais como a escolha aleatória de um Pokémon no início de uma batalha e após a derrota de um dos Pokémon do treinador. Outro método auxiliar a ser considerado é o retorno, caso o treinador ainda possua algum Pokémon disponível para batalha.

12) Deve possuir uma classe **Batalha**, que armazena os seguintes dados:

- Nome do treinador 1;
- Nome do treinador 2.
- Informações da batalha: um registro do que ocorre durante a batalha.

Obs.: Implemente um ou mais métodos para simular a batalha entre os treinadores. Porém, respeite as seguintes regras:

- A escolha dos pokémons iniciais deve ser aleatória;
- A definição da ordem de turno deve ser realizada com base na velocidade efetiva dos Pokémon, aleatória em caso de empate;
- Antes de realizar qualquer ação de turno, o Pokémon deve sofrer os efeitos de status com os quais ele está infligido;
- A batalha deve acabar somente quando um dos treinadores não possuir mais pokémons em condições de batalhar.

13) Deve possuir uma classe **Torneio**, contendo os seguintes dados:

- Os treinadores;
- O histórico de batalhas;
- O vencedor.

Obs.: Implemente um ou mais métodos para realizar o torneio. Se um treinador for derrotado, ele deve ser eliminado e o vencedor deve ser salvo em um atributo para facilitar o acesso.

- 14) É necessário implementar classes de erro para tratar operações incorretas.
- 15) Também é necessário reconhecer erros de tipos de dados inválidos ou incoerentes, bem como o cadastro de elementos duplicados.

Especificações e requisitos da aplicação:

- 1) A aplicação deve receber, como argumento na linha de comando, um arquivo de texto com as ações da aplicação para a realização de um torneio. Assim, o arquivo segue a seguinte lógica:
 - a) Como passo inicial do sistema, os tipos (golpe ou Pokémon) devem ser criados e deve-se obter a efetividade desses tipos em relação aos demais, por meio da leitura do arquivo especificado.
 - b) O segundo passo deve ser a criação dos Golpes, pois eles dependem do Tipo para serem criados.
 - c) Com estes dois sistemas criados, o Pokémon pode ser então instanciado, visto que ele depende tanto do sistema de Tipos quanto do de Golpes. Ele deve ter pelo menos 1 (um) golpe válido.
 - d) Com os Pokémon criados, podemos então definir os treinadores, já que eles devem obrigatoriamente possuir pelo menos 1 (um) Pokémon válido.
 - e) Além do sistema de Pokémon, o treinador também depende de um sistema de itens, mas este não é declarado no arquivo de entrada devido às suas nuances.
 - f) Então é declarado o Torneio, este contém os nomes de todos os Treinadores que participaram do mesmo.
 - g) Como última etapa, o Torneio é executado.
- 2) Após popular as classes com os dados do arquivo de texto recebido como argumento da linha de comando, a aplicação deverá executar o torneio e depois gerar os seguintes arquivos de saída:
 - a) **Torneio.out**: este log deve conter todos os dados do torneio, organizados por batalha e por turno. E, no final, deve dizer quem ganhou o torneio como um todo.
 - b) **Torneio.err**: este log deve conter todos os erros encontrados durante a execução, é importante que a aplicação não pare quando se deparar com ações não permitidas ou inválidas.
- 3) A aplicação não pode parar quando se deparar com ações/operações/transações não permitidas ou inválidas (ex.: treinadores estarem escritos mais de uma vez em um torneio, treinador possuir algum item em seu inventário que não está implementado no sistema, etc). Os erros devem ser armazenados em um arquivo de log.

Recomendação:

- Uma recomendação não obrigatória é utilizar a ideia de *Type Hint* do Python para declarar os tipos de retorno esperados nos métodos.
- Usar corretamente o Git, realizando commits e criando branches.

Relato

Faça um resumo de no máximo 400 palavras relatando as dificuldades, desafios ou outras



observações que considerar relevantes para a realização deste trabalho. Você também pode comentar sobre sua evolução no conhecimento. Esse resumo deve ser enviado junto com os demais arquivos no repositório do GitHub, no formato Markdown.

Entrega

Você deve enviar como pull request contendo o seguinte:

- **Torneio** (diretório do pacote)
- **Arquivos** (diretório onde devem estar os arquivos lidos e gerados)
- **app.py** (aplicativo que simula o sistema)
- **README.md** (arquivo descrevendo como usar o aplicativo e o pacote)
- **RESUMO.md** (arquivo contendo o resumo de 400 palavras)